

T01. Ciclo completo: registro > conexión > envío > desconexión > baja

<pre>###Terminal 1 – Servidor ./server -p 8888 s> init server 127.0.0.1:8888 s> # Al ejecutar T01: s> REGISTER alice OK s> CONNECT alice OK s> REGISTER bob OK s> CONNECT bob OK s> SEND MESSAGE 1 FROM alice TO bob s> DISCONNECT alice OK s> UNREGISTER alice OK</pre>	<pre>###Terminal 2 – Cliente Alice python3 client.py -s localhost -p 8888 c> REGISTER alice c> REGISTER OK c> CONNECT alice c> CONNECT OK c> SEND bob Hola bob! c> SEND OK – MESSAGE 1 # ACK llega tras entrega: c> SEND MESSAGE 1 OK c> DISCONNECT alice c> DISCONNECT OK c> UNREGISTER alice c> UNREGISTER OK</pre>	<pre>###Terminal 3 – Cliente Bob python3 client.py -s localhost -p 8888 c> REGISTER bob c> REGISTER OK c> CONNECT bob c> CONNECT OK # Mensaje llega automáticamente: s> MESSAGE 1 FROM alice Hola bob! END</pre>
---	--	---

T02. Mensajes encolados: envío a usuario offline > entrega al reconectar

<pre>###Terminal 1 – Servidor s> REGISTER alice OK s> REGISTER bob OK s> CONNECT alice OK # bob offline → almacenados: s> MESSAGE 1 FROM alice TO bob STORED s> MESSAGE 2 FROM alice TO bob STORED # bob conecta → entregados: s> CONNECT bob OK s> SEND MESSAGE 1 FROM alice TO bob s> SEND MESSAGE 2 FROM alice TO bob</pre>	<pre>###Terminal 2 – Alice (conectada) c> REGISTER alice c> CONNECT alice c> CONNECT OK c> SEND bob Mensaje 1 offline c> SEND OK – MESSAGE 1 c> SEND bob Mensaje 2 offline c> SEND OK – MESSAGE 2 #Cuando bob conecte, llegan los ACKs: c> SEND MESSAGE 1 OK c> SEND MESSAGE 2 OK</pre>	<pre>###Terminal 3 – Bob (offline → online) c> REGISTER bob c> REGISTER OK # bob desconectado, alice envía # (esperar unos segundos) c> CONNECT bob c> CONNECT OK # Recibe los 2 pendientes: s> MESSAGE 1 FROM alice Mensaje 1 offline END s> MESSAGE 2 FROM alice Mensaje 2 offline END</pre>
--	--	--

T03. USERS devuelve lista con formato "user :: IP :: puerto" y cache interna

	<pre>###Terminal 2 – Alice c> REGISTER alice c> CONNECT alice c> CONNECT OK c> USERS c> CONNECTED USERS (2 users connected) OK alice bob # _connected_users_list poblado internamente # con {alice:(ip,port), bob:(ip,port)}</pre>	<pre>###Terminal 3 – Bob c> REGISTER bob c> CONNECT bob c> CONNECT OK # Intentar USERS sin estar # conectado daría error: c> DISCONNECT bob c> USERS c> CONNECTED USERS FAIL, USER IS NOT CONNECTED</pre>
--	---	---

T04. Increment correcto de IDs: 3 mensajes secuenciales al mismo destinatario

	###Terminal 2 – Alice (envía) c> SEND bob Primero c> SEND OK – MESSAGE 1 c> SEND MESSAGE 1 OK c> SEND bob Segundo c> SEND OK – MESSAGE 2 c> SEND MESSAGE 2 OK c> SEND bob Tercero c> SEND OK – MESSAGE 3 c> SEND MESSAGE 3 OK # IDs 1,2,3: contador del remitente	###Terminal 3 – Bob (recibe) s> MESSAGE 1 FROM alice Primero END s> MESSAGE 2 FROM alice Segundo END s> MESSAGE 3 FROM alice Tercero END # Si ahora carol también envía a bob # el ID de carol empieza en 1 (independiente)
--	---	---

T05. Baja con mensajes pendientes: UNREGISTER elimina la cola de bob

	###Terminal 2 – Alice # alice conectada, bob offline c> SEND bob Msg que nunca llega c> SEND OK – MESSAGE 1 # Servidor almacena en pendientes de bob	###Terminal 3 – Bob # bob solo registrado, nunca conectado c> UNREGISTER bob c> UNREGISTER OK # Servidor borra el mensaje pendiente # Verificar: bob re-registrado y conectado # ya no recibe el mensaje anterior c> REGISTER bob c> CONNECT bob c> CONNECT OK # Sin mensajes en cola → silencio ✓
--	--	--

T06. Tres usuarios: alice > bob > carol, reenvios cruzados

###Terminal 2 – Alice c> REGISTER alice c> CONNECT alice c> SEND bob Hola bob c> SEND OK – MESSAGE 1 c> SEND carol Hola carol c> SEND OK – MESSAGE 2 c> SEND MESSAGE 1 OK c> SEND MESSAGE 2 OK	###Terminal 3 – Bob c> REGISTER bob c> CONNECT bob s> MESSAGE 1 FROM alice Hola bob END c> SEND carol Hola carol soy bob c> SEND OK – MESSAGE 1 c> SEND MESSAGE 1 OK	###Terminal 4 – Carol c> REGISTER carol c> CONNECT carol # recibe de alice y de bob: s> MESSAGE 2 FROM alice Hola carol END s> MESSAGE 1 FROM bob Hola carol soy bob END
--	--	---

T07. Doble error: CONNECT de no-registrado + doble CONNECT del mismo usuario

	###Terminal 2 – Un solo cliente c> CONNECT fantasma	
--	--	--

	<pre> c> CONNECT FAIL, USER DOES NOT EXIST c> REGISTER alice c> REGISTER OK c> CONNECT alice c> CONNECT OK c> CONNECT alice c> USER ALREADY CONNECTED c> DISCONNECT alice c> DISCONNECT OK c> DISCONNECT alice c> DISCONNECT FAIL, USER NOT CONNECTED c> UNREGISTER nadie c> USER DOES NOT EXIST </pre>	
--	--	--

T08. SEND mensaje de exactamente 255 caracteres y SEND a sí mismo

	<pre> ###Terminal 2 – Alice # Generar mensaje de 255 chars en bash: python3 -c "print('A'*255)" # Copiar el resultado y enviarlo: c> SEND bob AAAA...AAA c> SEND OK – MESSAGE 1 c> SEND MESSAGE 1 OK # SEND a sí misma: c> SEND alice Hola yo misma c> SEND OK – MESSAGE 2 # El hilo de escucha de alice # recibe el mensaje de alice s> MESSAGE 2 </pre>	<pre> ###Terminal 3 – Bob # Bob recibe los 255 chars intactos s> MESSAGE 1 FROM alice AAAAAAAAAAAAAAAAAAAAAAAA... (255 caracteres) END # Verificar longitud recibida: # debe ser exactamente 255 </pre>
--	---	--

T09. SEND a usuario inexistente y con servidor caído (robustez del cliente)

	<pre> ###Terminal 2 – Alice c> SEND noexiste Hola c> SEND FAIL, USER DOES NOT EXIST # Parar el servidor (Ctrl+C en Terminal 1) c> SEND bob Hola c> SEND FAIL # Cliente no crashea → sigue en el shell c> REGISTER nuevo c> REGISTER FAIL # Reiniciar servidor → operaciones OK </pre>	
--	---	--

T10. SENDATTACH completo: envío, recepcion y ACK con nombre de fichero

	<pre> ###Terminal 2 – Alice # Formato: SENDATTACH user mensaje fichero c> SENDATTACH bob Mira este fichero /tmp/datos.txt c> SENDATTACH OK – MESSAGE 1 # ACK al entregar: c> SEND MESSAGE 1 /tmp/datos.txt OK </pre>	<pre> ###Terminal 3 – Bob s> MESSAGE 1 FROM alice Mira este fichero END FILE /tmp/datos.txt # Ahora bob puede pedir el fichero c> GETFILE alice /tmp/datos.txt /tmp/recibido.txt c> GETFILE /tmp/datos.txt FROM alice OK # Verificar contenido: </pre>
--	---	---

T11. SENDATTACH encolado: destinatario offline > recibe adjunto al conectar

	<pre> ###Terminal 2 – Alice # bob offline, alice envía con fichero c> SENDATTACH bob Fichero offline /tmp/datos.txt c> SENDATTACH OK – MESSAGE 1 # Servidor almacena con has_file=1 # Cuando bob conecte, llegará el ACK: c> SEND MESSAGE 1 /tmp/datos.txt OK </pre>	<pre> ###Terminal 3 – Bob # bob conecta más tarde c> CONNECT bob c> CONNECT OK # Recibe mensaje + nombre fichero: s> MESSAGE 1 FROM alice Fichero offline END FILE /tmp/datos.txt c> GETFILE alice /tmp/datos.txt /tmp/local.txt c> GETFILE /tmp/datos.txt FROM alice OK </pre>
--	---	--

T12. GETFILE fichero binario: imagen o ejecutable transferido byte a byte

		<pre> ###Terminal 3 – Bob c> GETFILE alice /tmp/binario.bin /tmp/copia.bin c> GETFILE /tmp/binario.bin FROM alice OK # Verificar integridad byte a byte </pre>
--	--	--

T13. GETFILE con caché vacía y usuario desconectado entre medio

		<pre> ###Terminal 3 – Bob (sin haber hecho USERS antes) # caché _connected_users_list vacía c> GETFILE alice /tmp/datos.txt /tmp/local.txt # → internamente llama a USERS silencioso para refrescar # → alice encontrada → transferencia OK </pre>
--	--	---

		<pre> c> GETFILE /tmp/datos.txt FROM alice OK # Ahora alice se desconecta # bob intenta GETFILE de nuevo c> GETFILE alice /tmp/datos.txt /tmp/local2.txt # caché tiene alice pero está desconectada # → USERS silencioso → alice no aparece c> FILE TRANSFER FAILED, user not connected. </pre>
--	--	--

T14. Servicio web: normalización de espacios en SEND y SENDATTACH

<pre> ###Terminal 1 – Web service python3 web_service.py * Running on http://0.0.0.0:5000 # Verificar manualmente: curl "http://localhost:5000/normal ize?text=hola+++mundo" {"result":"hola mundo"} curl "http://localhost:5000/normal ize?text=++" {"result":""} </pre>	<pre> ###Terminal 2 – Alice (envío con espacios) # Mensaje con espacios repetidos c> SEND bob hola mundo que tal c> SEND OK – MESSAGE 1 # Bob recibe mensaje normalizado: # "hola mundo que tal" (1 espacio) c> SENDATTACH bob texto con espacios /tmp/f.txt c> SENDATTACH OK – MESSAGE 2 # Mensaje del adjunto también normalizado </pre>	
---	--	--

T15. Web service caído: cliente usa el mensaje original (fallback robusto)

	<pre> ###Terminal 2 – Alice (web service parado) # web_service.py NO está corriendo c> SEND bob hola mundo # Timeout de 2s al intentar conectar a :5000 # cliente captura la excepción → usa mensaje original c> SEND OK – MESSAGE 1 # Bob recibe "hola mundo" sin normalizar # El cliente NO crashea → comportamiento robusto ✓ </pre>	
--	---	--

T16. Servidor RPC: verifica que registra todas las operaciones

<pre> ###Terminal 1 – RPC server ./rpc_server s> Servidor RPC iniciado # Salida al ejecutar T16: alice REGISTER alice CONNECT bob REGISTER bob CONNECT alice SEND alice USERS alice SENDATTACH /tmp/datos.txt alice DISCONNECT alice UNREGISTER </pre>	<pre> ###Terminal 2 – Servidor mensajería LOG_RPC_IP=localhost ./server -p 8888 s> Conectado al servidor RPC en localhost s> init server 127.0.0.1:8888 </pre>	<pre> ###Terminal 3 – Cliente c> REGISTER alice c> CONNECT alice c> SEND bob Hola c> USERS c> SENDATTACH bob msg /tmp/datos.txt c> DISCONNECT alice c> UNREGISTER alice # Cada cmd aparece en Terminal 1 ✓ </pre>
---	--	--

T17. Servidor sin LOG_RPC_IP: funciona sin RPC (sin crash)

<pre> ###Terminal 1 – Servidor sin variable de entorno ./server -p 8888 # Sin LOG_RPC_IP → aviso en stderr, continúa: s> Aviso: LOG_RPC_IP no definida, sin registro RPC s> init server 127.0.0.1:8888 # Todas las operaciones funcionan igual # Solo sin el log RPC → comportamiento degradado esperado </pre>		
---	--	--

T18. Concurrencia: 20 registros distintos simultaneos > todos OK sin corrupción

	<pre> ###Terminal 2 – Script de concurrency python3 test_concurrencia.py -s localhost -p 8888 # Suite C01: [PASS] Todos los registros devuelven 0 (OK) → 20/20 OK, 0 fallos [PASS] Sin respuestas perdidas → respuestas: 0 nulas # Suite C02 (mismo nombre): [PASS] Exactamente 1 registro OK (mutex) → 1 OK, 9 duplicados [PASS] Resto devuelve 1 (USERNAME IN USE) # Detecta race condition en find_user() # si el mutex no funciona → más de 1 OK </pre>	
--	---	--

T19. Concurrencia: 10 SEND simultáneos al mismo destinatario > IDs únicos

	<pre> ###Terminal 2 – Script de concurrency (suite C04) python3 test_concurrencia.py -s localhost -p 8888 </pre>	
--	--	--

	<pre># Suite C04: [PASS] Todos los SEND devuelven código 0 → 10/10 enviados [PASS] IDs de mensaje únicos (sin race condition) → 10 únicos de 10 [PASS] Bob recibe todos los mensajes → 10/10 en listener # Si msg_counter no está bajo mutex: # dos hilos pueden leer el mismo valor → IDs duplicados → FAIL</pre>	
--	--	--

T20. Concurrencia: estrés 50 operaciones en ráfaga + conexiones/desconexiones

	<pre>###Terminal 2 – Script completo (todas las suites) python3 test_concurrencia.py -s localhost -p 8888 # Suites C05 (conexiones/desconexiones simultáneas): [PASS] Desconexiones simultáneas sin deadlock → 4/4 OK [PASS] Sin respuestas inválidas (servidor estable) # Suite C07 (estrés 50 operaciones): [PASS] Servidor responde a las 50 operaciones → 0 timeouts [PASS] Solo códigos válidos 0 o 1 → sin corrupción Resultado: 16/17 passed Tiempo total: 3.42s # Si hay FAILs → problema real de concurrencia</pre>	
--	---	--

